CSRF Security Testing Guide

Educational Bug Bounty Series • Authorized Security Research Only

■ Cross-Site Request Forgery (CSRF) — Security Testing Guide

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Perform checks only on authorized applications with controlled accounts. Use harmless, reversible test actions and never create a page designed to change another real user's account.*

■ Executive Summary

Cross-Site Request Forgery (CSRF) abuses a browser's existing authenticated state to cause an action the user did not intend. The supplied four-page *CSRF 2.pdf* introduces CSRF, shows GET/POST and JSON-style examples, and discusses multipart requests and token-validation edge cases.

■ Core Testing Methodology

1. **Find state-changing endpoints.** Prioritize profile, email, password, and other sensitive changes.
2. **Record normal behavior.** Capture a request from your own test session and identify its authentication and CSRF controls.
3. **Check methods and content types.** Verify that each supported request format receives the same appropriate protection.
4. **Verify token enforcement.** Check that absent, invalid, expired, or session-mismatched tokens are rejected.
5. **Check token binding.** Confirm tokens are associated with the right user/session and cannot be substituted across test accounts.
6. **Review multipart handling.** Ensure file-upload or multipart forms do not bypass normal CSRF validation.
7. **Check token validation logic.** Presence-only checks and ambiguous token comparisons are insufficient.
8. **Review browser controls.** Confirm cookie `SameSite` settings and origin validation align with the application's needs.
9. **Verify actual state.** A successful HTTP response alone does not prove the action occurred; inspect only your test account.
10. **Document and retest.** Keep evidence redacted and repeat the test after the fix.

■ High-Value Bug Bounty Tips & Edge Cases

- **GET and POST:** Sensitive state changes should not be reachable through a safe method such as GET.
- **JSON requests:** CORS and content-type behavior may affect exploitability, but neither should be treated as the sole CSRF defense.
- **Multipart requests:** Apply CSRF controls to multipart and upload workflows.
- **Token variations:** Check missing, malformed, expired, and session-mismatched tokens in a lab.
- **Duplicate token/cookie patterns:** Validate the intended relationship between the submitted token and cookie; do not accept arbitrary matching values without appropriate integrity.
- **Source limitation:** The source contains proof-of-concept markup and scripts. This rewrite omits deployable cross-site attack code and does not claim a specific application is vulnerable.

■■ Educational Checklists & Safe Examples

```
Endpoint: [In-Scope-State-Changing-Endpoint]
Method/content type: [Documented Format]
CSRF control: [Token / Origin / Framework Middleware]
Test case: [Missing / Invalid / Expired / Cross-Session]
Expected: [Request Rejected]
Observed: [Redacted]
```

```
State changed on test account: [Yes/No]
```

- [ ] CSRF protections cover all state-changing endpoints.
- [ ] Tokens are unpredictable and session-bound.
- [ ] Multipart and alternate content types are covered.
- [ ] Origin checks and cookie policies are configured.
- [ ] GET does not change sensitive state.
- [ ] Invalid requests do not change account state.

■■ Mitigation & Remediation

- Use maintained framework CSRF middleware and correctly validate tokens server-side.
- Apply protection to all state-changing endpoints, including multipart requests.
- Bind tokens to the user session and reject missing, invalid, or expired tokens.
- Validate `Origin` and `Referer` as defense in depth.
- Use `SameSite` cookies where compatible with the application.
- Avoid state changes through GET and test CORS configuration separately.
- Add automated regression tests for token and method edge cases.

■ Source Notes

Based on the supplied four-page *CSRF 2.pdf*. Page 1 defines CSRF and shows GET, POST, and JSON request patterns. Page 2 contains a multipart example and introduces CSRF-token bypass categories; the later pages have limited readable text in the supplied preview. The rewrite preserves the visible subject areas and replaces operational attack code with a safe test record.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.